

LAPORAN PRAKTIKUM
STRUKTUR DATA
“laporan Praktikum Pekan 2”

Disusun Oleh:

Faiz Fikri Satria

2511533026

Dosen Pengampu : Dr. Wahyudi, S.T, M.T.
Asisten Praktikum : Sherly Sukmadira



DAPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas rahmat-Nya sehingga laporan praktikum pertemuan kedua ini dapat diselesaikan dengan baik. Laporan ini mendokumentasikan implementasi struktur data linear menggunakan kelas bawaan Java, yaitu ArrayList, serta pengembangannya dalam objek kustom. Penulis menyadari laporan ini masih memiliki kekurangan dan mengharapkan kritik serta saran yang membangun.

Padang 19 April, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat	2
BAB II PEMBAHASAN	3
2.1 Implementasi Dasar ArrayList	3
2.1.1 Langkah-langkah	3
2.1.2 Contoh Pemrograman	3
2.1.3 Analisis	4
2.2 Enkapsulasi Daftar Kata	4
2.2.1 Langkah-langkah	4
2.2.2 Contoh Pemrograman	5
2.2.3 Hasil Output	5
2.2.4 Analisis	5
2.3 Model Objek Kustom	6
2.2.1 Langkah-langkah	6
2.3.2 Contoh Pemrograman	6

2.3.3 Hasil Output	6
2.3.4 Analisis	6
2.4 Sistem Driver Terintegrasi	7
2.4.1 Langkah-langkah	7
2.4.2 Contoh Pemrograman	7
2.4.3 Hasil Output	7
2.4.4 Analisis	7
2.5 Tugas Pekan 2	9
2.5.1. Analisis	8
BAB III KESIMPULAN	10
DAFTAR PUSTAKA	11

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman Java, pengelolaan sekumpulan data merupakan kebutuhan yang sangat fundamental. Pada tahap awal, penggunaan *array* statis sering menjadi pilihan utama karena akses datanya yang cepat. Namun, *array* statis memiliki keterbatasan besar, yaitu ukurannya yang tetap (*fixed-size*) dan tidak dapat diubah setelah dideklarasikan. Hal ini menimbulkan inefisiensi memori jika data yang disimpan jauh lebih sedikit dari kapasitasnya, atau justru menyebabkan error jika data melebihi kapasitas yang telah ditentukan sebelumnya.

Untuk mengatasi batasan tersebut, Java menyediakan `ArrayList` sebagai bagian dari *Collections Framework*. `ArrayList` adalah struktur data berbasis *dynamic array* yang memungkinkan penambahan dan penghapusan elemen secara fleksibel tanpa harus mendefinisikan ukuran maksimum di awal. Selain fleksibilitas ukuran, `ArrayList` juga menyediakan berbagai *method* bawaan untuk manipulasi data seperti pencarian, pengurutan, dan penyisipan yang lebih mudah dibandingkan *array* biasa. Pemahaman mendalam mengenai `ArrayList` menjadi sangat penting bagi mahasiswa informatika karena struktur ini merupakan fondasi utama dalam membangun aplikasi yang mampu mengelola data dinamis secara efisien dan sistematis.

1.2 Tujuan

1. Memahami penggunaan `java.util.ArrayList` sebagai implementasi dari *dynamic array*.
2. Mampu melakukan operasi dasar seperti menambah, menghapus, mengubah, dan mengakses elemen berdasarkan indeks.
3. Menguasai teknik iterasi pada koleksi data menggunakan loop tradisional maupun *for-each*.
4. Mampu mengintegrasikan `ArrayList` dengan objek buatan sendiri (Custom Class) untuk manajemen data yang kompleks.

1.3 Manfaat

1. Memberikan pemahaman tentang fleksibilitas ukuran data dibandingkan dengan array statis.
2. Meningkatkan kemampuan dalam merancang sistem manajemen informasi sederhana (CRUD) berbasis memori.
3. Sebagai dasar untuk memahami struktur data yang lebih lanjut seperti *Linked List* dan *Stack/Queue*.

BAB II

PEMBAHASAN

2.1 Implementasi Dasar ArrayList

2.1.1 Teori

ArrayList adalah bagian dari Java Collections Framework yang mengimplementasikan interface List. Berbeda dengan array standar yang memiliki ukuran tetap, ArrayList dapat mengubah ukurannya secara dinamis saat elemen ditambahkan atau dihapus. Ia menggunakan array internal untuk menyimpan elemen dan secara otomatis mengalokasikan memori baru ketika kapasitasnya penuh.

2.1.2 Langkah-Langkah

1. Melakukan import java.util.ArrayList.
2. Mendeklarasikan objek ArrayList dengan tipe data Integer.
3. Menambahkan elemen menggunakan perulangan dan method add().
4. Menghapus elemen tertentu menggunakan indeks melalui method remove().
5. Menampilkan isi list sebelum dan sesudah manipulasi.

2.1.3. Contoh Pemrograman

```
1 package pekan2_2511533026;
2 import java.util.ArrayList;
3
4 public class ArrayList1_2511533026 {
5     public static void main(String[] args) {
6         //Size of the ArrayList
7         int n = 5;
8         // Declaring the ArrayList with initial size n
9         ArrayList<Integer> arrli = new ArrayList<Integer>(n);
10        for (int i = 1; i <= n; i++)
11            arrli.add(i);
12        //Printing elements
13        System.out.println(arrli);
14        //Remove element at index 3
15        arrli.remove(3);
16        //Displaying the ArrayList
17        //after deletion
18        System.out.println(arrli);
19        //Printing elements one by one
20        for (int i = 0; i < arrli.size(); i++)
21            System.out.print(arrli.get(i) + " ");
22    }
23 }
```

2.1.4. Analisis

Pada kode ini, saat `arrli.remove(3)` dipanggil, elemen bernilai 4 dihapus karena berada pada indeks ke-3 (indeks dimulai dari 0). Ukuran list berkurang secara dinamis, dan elemen 5 bergeser menempati indeks ke-3.

2.2 Enkapsulasi Daftar Kata

2.2.1 Teori

Enkapsulasi dalam struktur data bertujuan untuk menyembunyikan detail implementasi penyimpanan. Dengan membungkus `ArrayList` di dalam kelas `DaftarKata`, kita dapat mengontrol bagaimana data diakses dan dimodifikasi melalui method-method tertentu (setter/getter kustom).

2.2.2. Langkah-Langkah

1. Membuat atribut privat `ArrayList<String>`.
2. Membuat method `tambah()` untuk menyisipkan data di akhir.
3. Membuat method `ubahElemen()` yang memanfaatkan fungsi `set()`.
4. Membuat mekanisme iterasi untuk mencetak data secara terstruktur.

2.2.3. Contoh Pemrograman

```
1 package pekan2_2511533026;
2 import java.util.ArrayList;
3 public class DaftarKata_2511533026 {
4     private final ArrayList<String> data;
5     // Konstruktor: inisialisasi list kosong
6     public DaftarKata_2511533026() {
7         this.data = new ArrayList<>();
8     }
9     /** Menambahkan elemen di akhir list. */
10    public void tambah (String elemen) {
11        data.add(elemen);
12    }
13    /** Menambahkan elemen pada indeks tertentu (memvisipkan). */
14    public void tambahPada(int index, String elemen) {
15        data.add(index, elemen);
16    }
17    /**
18     * Mengubah elemen pada posisi 'index' menjadi 'nilaiBaru'. elemen
19     * Bertindak sebagai "setter" untuk elemen tertentu.
20     */
21    public void ubahElemen(int index, String nilaiBaru) {
22        data.set(index, nilaiBaru);
23    }
24    /**
25     * Menghapus elemen pada posisi 'index' dan mengembalikan nilai yang dihapus.
26     */
27    public String hapusElemen(int index) {
28        return data.remove(index);
29    }
30    /**
31     * Melakukan iterasi dan mencetak setiap elemen dalam format: [index] nilai
32     * (Metode ini tidak mengembalikan nilai; hanya demonstrasi iterasi).
33     */
34
35    public void iterasiCetak() {
36        for (int i = 0; i < data.size(); i++) {
37            System.out.print(data.get(i)+" ");
38        }
39    }
40    /** Mengambil elemen berdasarkan indeks. */
41    public String get (int index) {
42        return data.get(index);
43    }
44
45    // Representasi string agar mudah dicetak
46    @Override
47    public String toString() {
48        return data.toString();
49    }
50 }
```

2.2.4 Hasil Output

```
Console X
<terminated> DaftarKataDriver_2511533026 [Java Application] C:\Users\Lenovo\p2
Awal : [Kami, Mahasiswa, Informatika]
Setelah Ubah: [Kami, Departemen, Informatika]
Terhapus : Kami
Setelah Hapus: [Departemen, Informatika]
Iterasi: Departemen Informatika
```

2.2.5 Analisis

Method set() pada ArrayList digunakan untuk mengganti nilai pada indeks tertentu tanpa mengubah ukuran list. Penggunaan *for-each loop* dalam method cetak memudahkan akses seluruh elemen tanpa perlu mengelola variabel counter manual.

2.3 Model Objek Kustom

2.3.1. Teori

ArrayList tidak terbatas pada tipe data primitif/wrapper saja, tetapi juga dapat menyimpan referensi ke objek kustom. Dalam hal ini, kelas Mahasiswa berfungsi sebagai *Data Transfer Object* (DTO) yang mengelompokkan beberapa atribut terkait menjadi satu kesatuan.

2.3.2. Langkah-Langkah

1. Mendefinisikan atribut nim, nama, dan prodi.
2. Membuat Constructor untuk pengisian data awal objek.
3. Melakukan *Override* pada method toString() dari kelas Object untuk mendefinisikan tampilan teks dari objek tersebut.

2.3.3. Contoh Pemrograman

```
1 package pekan2_2511533026;  
2  
3 public class Mahasiswa_2511533026 {  
4     String nim;  
5     String nama;  
6     String prodi;  
7     Mahasiswa_2511533026(String nim, String nama, String prodi) {  
8         this.nim = nim;  
9         this.nama = nama;  
10        this.prodi = prodi;  
11    }  
12    @Override  
13    public String toString() {  
14        return "NIM: " + nim + ", Nama: " + nama + ", Prodi: " + prodi;  
15    }  
16 }  
17
```

2.3.4. Analisis

Tanpa @Override toString(), saat kita mencetak objek mahasiswa, Java hanya akan menampilkan alamat memori (hashcode). Dengan meng-override-nya, kita memberikan instruksi spesifik kepada program untuk menampilkan data atribut yang relevan.

2.4 Sistem Driver Terintegrasi

2.4.1. Teori

Kelas Driver berfungsi sebagai *controller* yang menghubungkan antarmuka pengguna (Console) dengan logika penyimpanan data. Di sini, ArrayList digunakan sebagai database sementara (in-memory) untuk mengelola sekumpulan objek Mahasiswa.

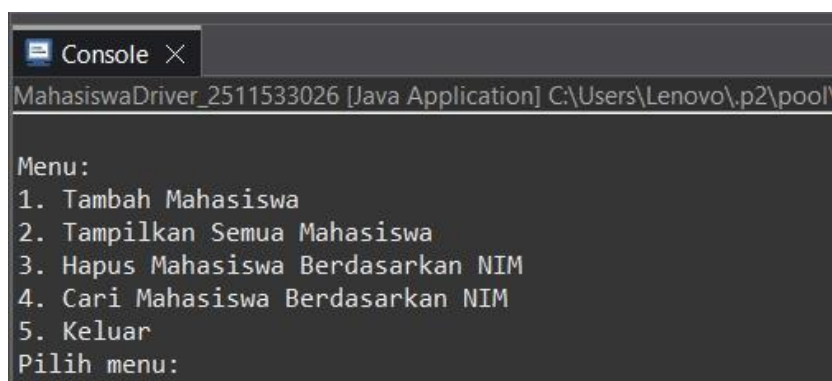
2.4.2. Langkah-Langkah

1. Membuat ArrayList<Mahasiswa_2511533026>.
2. Menggunakan Scanner untuk menerima input user dalam perulangan.
3. Menggunakan method `removeIf()` dengan ekspresi *lambda* untuk penghapusan data berdasarkan kriteria NIM.

2.4.3. Contoh Pemrograman

```
1 package pkaniz_2511533026;
2
3 import java.util.ArrayList;
4 public class MahasiswaDriver_2511533026 {
5     // Method untuk menampilkan menu
6     public static void tampilMenu() {
7         System.out.println("Menu:");
8         System.out.println("1. Tambah Mahasiswa");
9         System.out.println("2. Tampilkan Semua Mahasiswa");
10        System.out.println("3. Hapus Mahasiswa Berdasarkan NIM");
11        System.out.println("4. Cari Mahasiswa Berdasarkan NIM");
12        System.out.println("5. Keluar");
13    }
14
15    // Method untuk Tambah Mahasiswa
16    public static void tambahMahasiswa(ArrayList<Mahasiswa_2511533026> list, Scanner sc) {
17        System.out.print("Masukkan NIM: ");
18        String nim = sc.nextLine();
19        System.out.print("Masukkan Nama: ");
20        String nama = sc.nextLine();
21        System.out.print("Masukkan Prodi: ");
22        String prodi = sc.nextLine();
23        list.add(new Mahasiswa_2511533026(nim, nama, prodi));
24        System.out.println("Mahasiswa berhasil ditambahkan");
25    }
26
27    // Method untuk Tampilkan Semua data
28    public static void tampilSemuaMahasiswa(ArrayList<Mahasiswa_2511533026> list) {
29        if (list.isEmpty()) {
30            System.out.println("Tidak ada mahasiswa yang");
31        } else {
32            System.out.println("Data Mahasiswa:");
33            for (Mahasiswa_2511533026 mhs : list) {
34                System.out.println(mhs);
35            }
36        }
37    }
38 }
```

2.4.4. Hasil Output



```
Console X
MahasiswaDriver_2511533026 [Java Application] C:\Users\Lenovo\p2\pool\

Menu:
1. Tambah Mahasiswa
2. Tampilkan Semua Mahasiswa
3. Hapus Mahasiswa Berdasarkan NIM
4. Cari Mahasiswa Berdasarkan NIM
5. Keluar
Pilih menu:
```

2.4.5. Analisis

Method `removeIf()` adalah fitur efisien dalam Java untuk menghapus elemen yang memenuhi syarat tertentu tanpa perlu melakukan iterasi manual yang rentan terhadap `ConcurrentModificationException`. Ini menunjukkan penggunaan `ArrayList` dalam skenario aplikasi nyata.

2.5 Tugas Pekan 2

2.5.1 Analisis

Kode tugas menunjukkan penggunaan metode manipulasi list yang lebih mendalam, di mana penyisipan data dilakukan di posisi tengah list menggunakan `add(index, value)`. Hal ini memperlihatkan bagaimana `ArrayList` secara internal mengelola pergeseran memori (*memory shifting*). Selain itu, penggunaan tipe data `String` pada tugas ini menunjukkan kemampuan list dalam menangani data tekstual secara dinamis tanpa perlu menentukan ukuran array di awal program.

```
1 package Pekan2_2511533026;
2 import java.util.ArrayList;
3
4
5 public class Playlist_2511533026 {
6     public static void main(String[] args) {
7         ArrayList<Musik_2511533026> playlist = new ArrayList<>();
8         Scanner sc = new Scanner(System.in);
9
10        while (true) {
11            System.out.println("=== Playlist Musik ===");
12            System.out.println("1. Tambah Lagu");
13            System.out.println("2. Lihat Playlist");
14            System.out.println("3. Hapus Lagu");
15            System.out.println("4. Keluar");
16            System.out.print("Pilihan: ");
17            int pilihan = sc.nextInt();
18            sc.nextLine(); // membersihkan buffer
19
20            switch (pilihan) {
21                case 1: // Tambah Lagu
22                    System.out.print("Masukkan Judul: ");
23                    String judul = sc.nextLine();
24                    System.out.print("Masukkan Penyanyi: ");
25                    String penyanyi = sc.nextLine();
26                    System.out.print("Masukkan Durasi (menit): ");
27                    int durasi = sc.nextInt();
28                    sc.nextLine();
29
30                    Musik_2511533026 laguBaru = new Musik_2511533026(judul, penyanyi, durasi);
31                    playlist.add(laguBaru);
32                    System.out.println("Data berhasil ditambahkan!");
33                    System.out.println();
34            }
35        }
36    }
37 }
```

```

1 package Playlist_2511533026;
2
3 public class Musik_2511533026 {
4     private String judul_3026;
5     private String penyanyi_3026;
6     private int durasi_3026;
7
8     // Constructor
9     public Musik_2511533026(String judul_3026, String penyanyi_3026, int durasi_3026) {
10         this.judul_3026 = judul_3026;
11         this.penyanyi_3026 = penyanyi_3026;
12         this.durasi_3026 = durasi_3026;
13     }
14
15     // Getter (Sisikluu)
16     public String getJudul_3026() {
17         return judul_3026;
18     }
19
20     public String getPenyanyi_3026() {
21         return penyanyi_3026;
22     }
23
24     public int getDurasi_3026() {
25         return durasi_3026;
26     }
27
28     // Setter (Mutator)
29     public void setJudul_3026(String judul_3026) {
30         this.judul_3026 = judul_3026;
31     }
32
33     public void setPenyanyi_3026(String penyanyi_3026) {
34         this.penyanyi_3026 = penyanyi_3026;
35     }
36
37     public void setDurasi_3026(int durasi_3026) {
38         this.durasi_3026 = durasi_3026;
39     }
40 }

```

```

Console X
Playlist_2511533026 [Java Application] D:\eclipse\plugins\org
=== Playlist Musik ===
1. Tambah Lagu
2. Lihat Playlist
3. Hapus Lagu
4. Keluar
Pilihan:

```

Analisis Hasil: Berdasarkan hasil eksekusi, setiap perubahan yang dilakukan pada list (baik itu penambahan, perubahan nilai, maupun penghapusan) langsung tercermin pada indeks elemen lainnya. Saat satu elemen dihapus, total size() dari koleksi berkurang secara otomatis, dan elemen terakhir akan mengisi posisi indeks sebelumnya, sehingga integritas urutan data tetap terjaga tanpa adanya celah kosong di tengah list.

BAB III

KESIMPULAN

Praktikum pada pekan kedua ini memberikan pemahaman mendalam mengenai fleksibilitas ArrayList dibandingkan dengan array statis konvensional. Melalui serangkaian implementasi mulai dari tipe data dasar hingga objek kustom, dapat disimpulkan bahwa ArrayList merupakan struktur data linear yang sangat efisien untuk menangani sekumpulan data yang jumlahnya tidak pasti atau sering berubah-ubah. Penguasaan terhadap method-method penting seperti add, set, remove, dan removeIf memungkinkan pengembang untuk membangun sistem manajemen data yang dinamis dengan kode yang lebih ringkas dan terorganisir.

Selain itu, integrasi antara struktur data ArrayList dengan prinsip Pemrograman Berorientasi Objek (OOP) seperti enkapsulasi dan penggunaan kelas model terbukti sangat efektif dalam membangun aplikasi yang modular. Dengan memisahkan logika penyimpanan di kelas driver dan representasi data di kelas model mahasiswa, kode menjadi lebih mudah untuk dikembangkan dan dipelihara. Kemampuan untuk melakukan manipulasi data berdasarkan kondisi tertentu melalui ekspresi lambda menunjukkan bahwa Java menyediakan perkakas yang kuat untuk mengelola koleksi objek secara kompleks namun tetap memiliki performa yang optimal.

DAFTAR PUSTAKA

- a. Oracle. (2023). *ArrayList (Java SE 17 & JDK 17)*.
- b. Goodrich, M. T., et al. (2014). *Data Structures and Algorithms in Java*. Wiley.
- c. Materi Kuliah Struktur Data – Universitas Andalas.